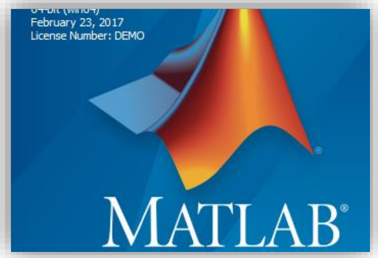
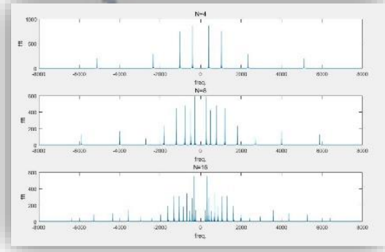
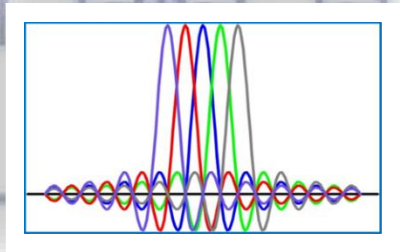
信号与系统实验

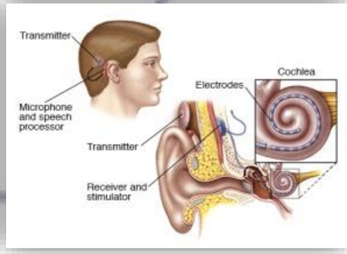
主讲人：吴光 博士

Email: wug@sustech.edu.cn









Signals and Systems (Lab)

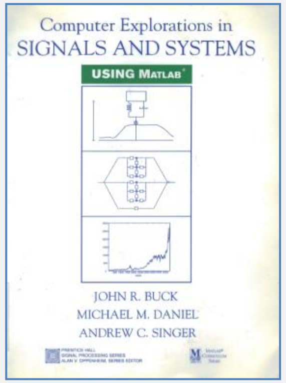
Lab 5: Butterworth Filter Design

Dr. Wu Guang
wug@sustech.edu.cn
Electrical & Electronic Engineering
Southern University of Science and Technology

Labs in this course

5 Lab assignments+2 Projects

- Lab 1: MATLAB Programming (3 Weeks)
- Lab 2: Linear Time-Invariant Systems (2 Weeks)
- Lab 3: Fourier Series Representation of Periodic Signals (2 Weeks)
- **Lab 4: The Continuous-Time Fourier Transform (2 Weeks)**
- Lab 5: Coding Test & System, Transform, Convolution and Filter (1 Week)
- Project 1: Speech synthesis and perception with envelope cue (3 Weeks)
- Project 2: Motion detection via communication signals (3 Weeks)



Review—Use fft() to calculate CTFT approximately

- Suppose the dominant region of x(t) is in [0,T], then we can use the following approximation

> - $X(j\omega)=\int_{-\infty}^{\infty} x(t)e^{-j\omega t}dt \approx \int_0^T x(t)e^{-j\omega t}dt \approx \sum_{n=0}^{N-1} x(n\tau)e^{-j\omega n\tau}\tau$

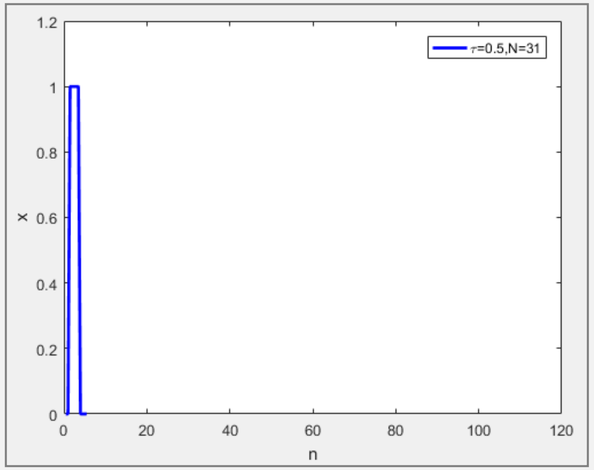
- It could be observed that
  - Slice into N intervals, each with length τ=T/N

> - $X(j\frac{2\pi k}{N\tau})\approx \boxed{\tau\sum_{n=0}^{N-1} x(n\tau)e^{-j\frac{2\pi kn}{N}}}$

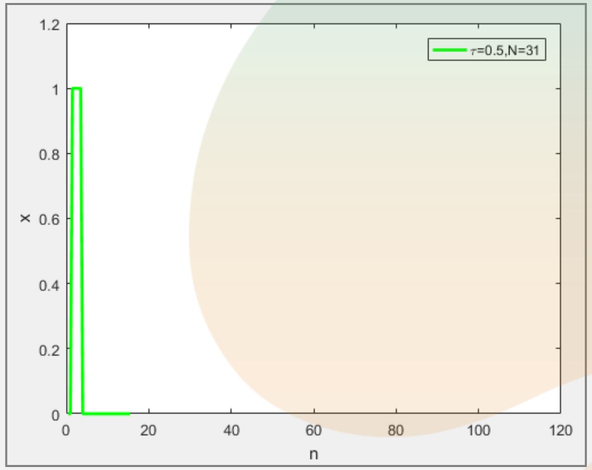
  - $k=\frac{1-N}{2},\ldots,-1,0,1,\ldots,\frac{N-1}{2}$
  - fft() of [x(0), $x(\tau)$, $x(2\tau)$, …, x((N-1)τ)]

- **Conclusion**: we can use fft() to calculate CTFT approximately, which could reduce the computation complexity

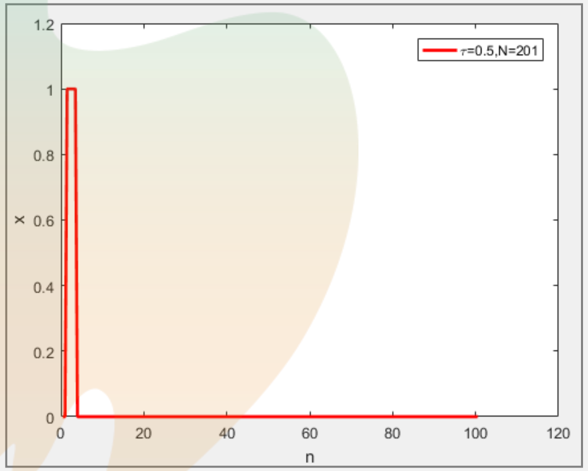
Review—Increase N



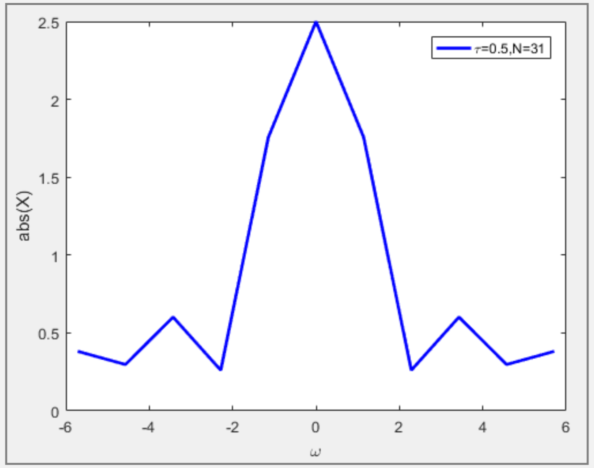
N=11

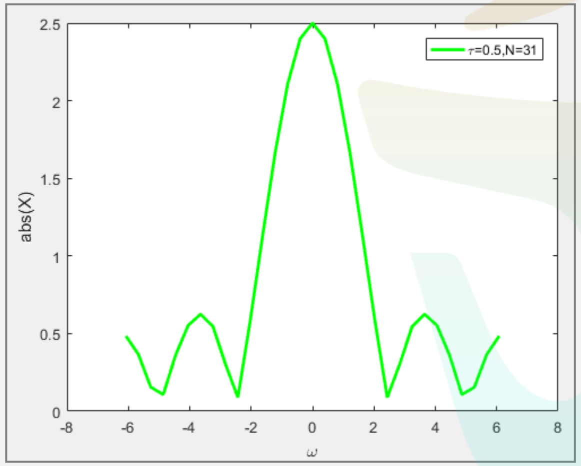


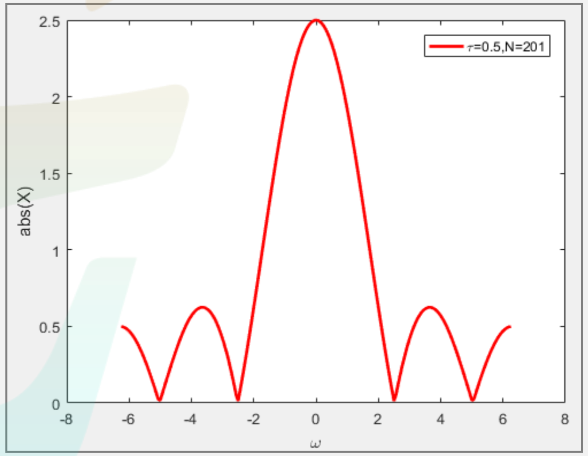
N=31



N=201







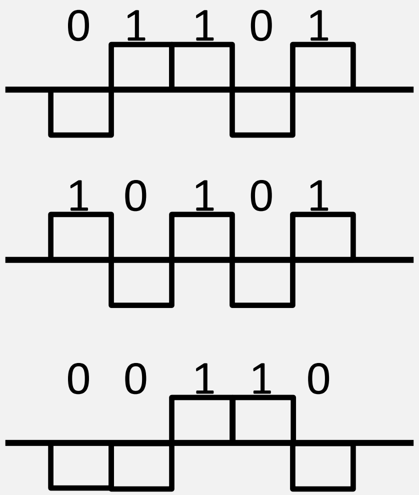
Review—Frequency Response and Impulse Response

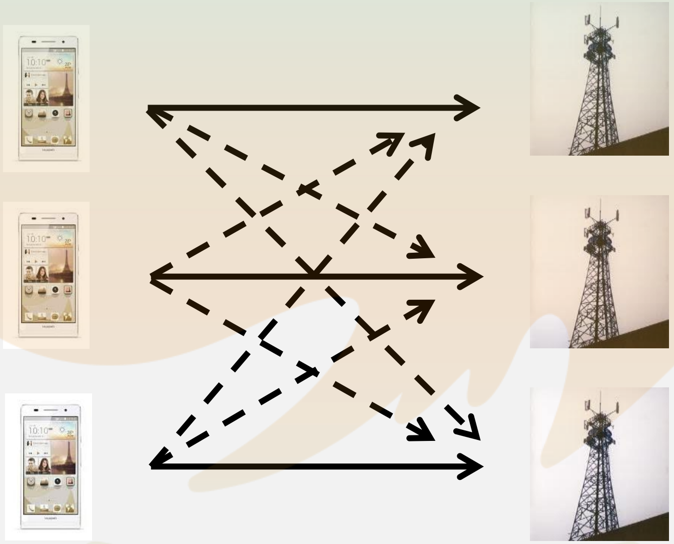
Frequency Response $\xLeftrightarrow{\text{Fourier Transform}}$ **Impulse Response**

- **Transform pair:**
  - $e^{-at}u(t) \quad Re\{a\} > 0 \quad \Leftrightarrow \quad \frac{1}{j\omega+a}$
- **Example:**
  - $\frac{3}{j\omega+3} \Leftrightarrow \quad 3e^{-3t}u(t)$
- **How about general fraction?**

$$H(j\omega) = \frac{b_M(j\omega)^M + b_{M-1}(j\omega)^{M-1} + \cdots + b_1(j\omega) \quad + b_0}{a_N(j\omega)^N + a_{N-1}(j\omega)^{N-1} + \cdots + a_1(j\omega) \quad + a_0}$$

Review—Application：What is Modulation？

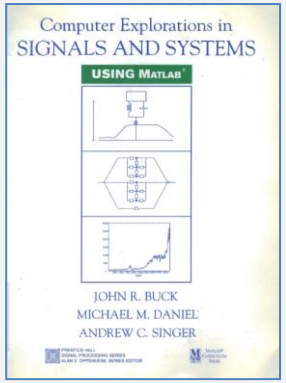




- Each phone wants to deliver information to its base station
- Therefore, there is cross-talk in the wireless channel
- How can we solve this issue?

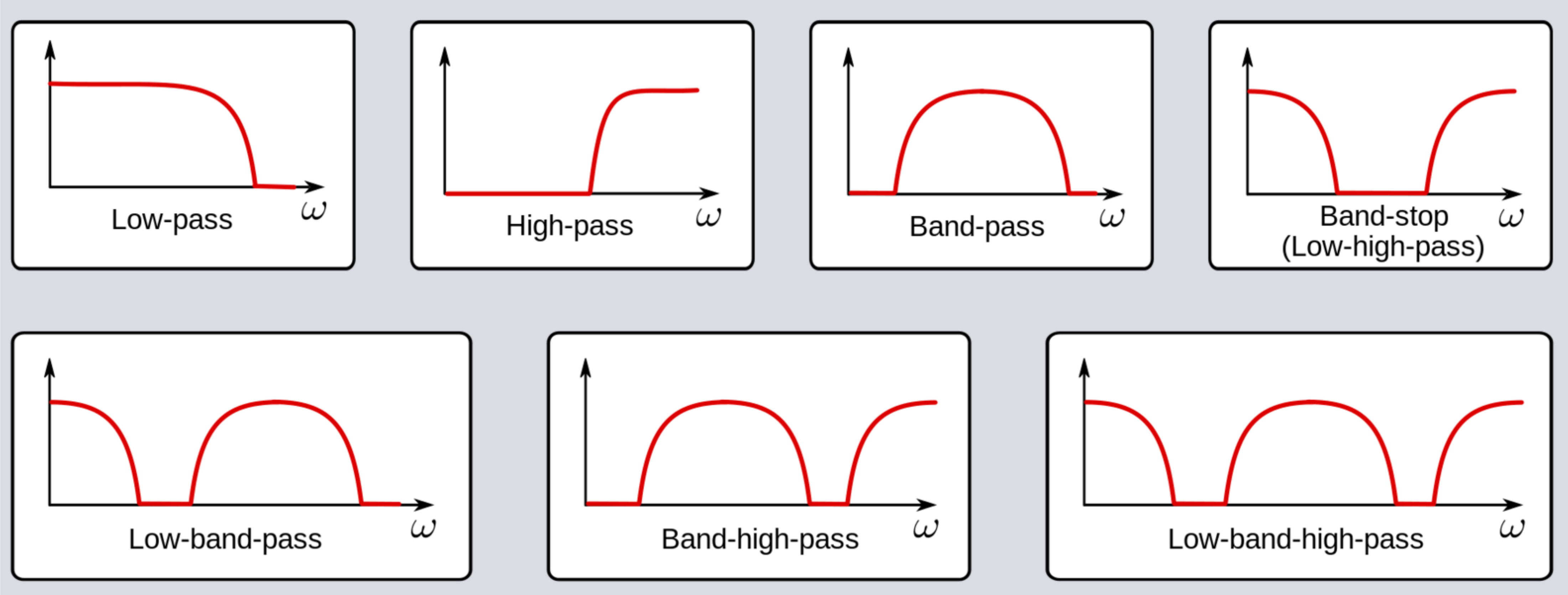
Labs in this course

5 Lab assignments+2 Projects



- Lab 1: MATLAB Programming (3 Weeks)
- Lab 2: Linear Time-Invariant Systems (2 Weeks)
- Lab 3: Fourier Series Representation of Periodic Signals (2 Weeks)
- Lab 4: The Continuous-Time Fourier Transform (2 Weeks)
- **Lab 5: Coding Test & System, Transform, Convolution and Filter (1 Week)**
- Project 1: Speech synthesis and perception with envelope cue (3 Weeks)
- Project 2: Motion detection via communication signals (3 Weeks)

Question：What is filter？



1. Butterworth filter design

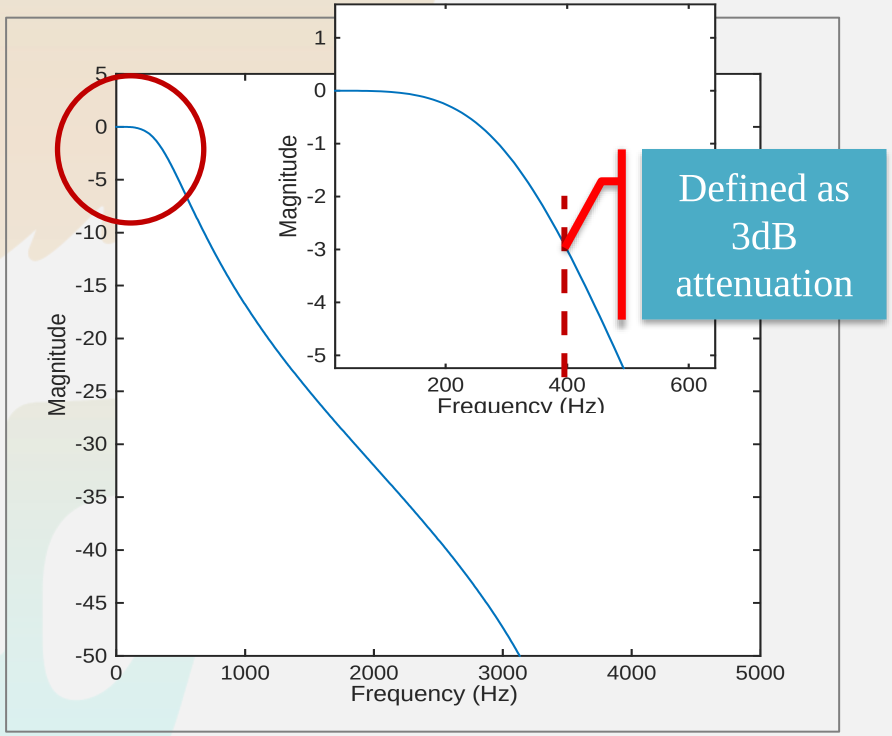
- **[b, a]=butter(n, Wn)**
  - Design an order n low-pass digital Butterworth filter with ***normalized cutoff frequency*** Wn. Wn must be 0.0 < Wn < 1.0, with 1.0 corresponding to half the sample rate.
  - Return the filter coefficients in length n+1 vectors b and a.
- **[b, a]=butter(n, Wn, 'ftype')**
  - Design a highpass, lowpass, or bandstop filter, where the string ‘ftype’ is ‘high’, ‘low’, or ‘stop’,
  - e.g.,

```
fs=8000; [b, a]=butter(2,400/(fs/2));        % low-pass
[b, a]=butter(2,400/(fs/2),'high');          % high-pass
[b, a]=butter(2,[400 1000]/(fs/2));          % band-pass
```

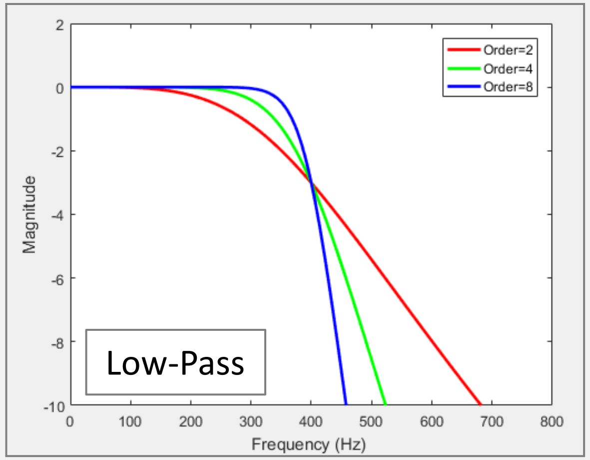
Exercise 1

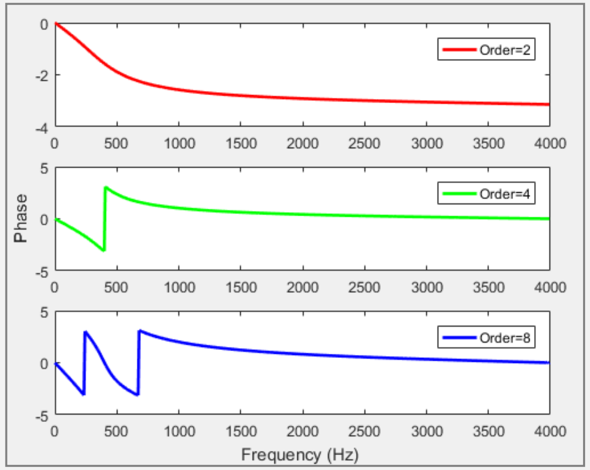
```
fs=8000;                        % sampling rate for discrete signal
[b, a]=butter(2,400/(fs/2));
[h,f]=freqz(b,a,512,fs);        % Digital filter frequency response
plot(f,20*log10(abs(h)));       % in dB scale
axis([0 5000 -50 5]);
xlabel('Frequency (Hz) ');
ylabel('Magnitude');
```

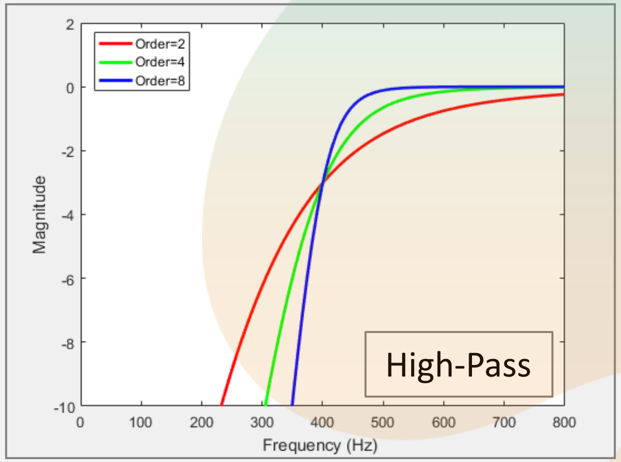
How about ‘high-pass’ and ‘band-pass’?
How about high-order n?

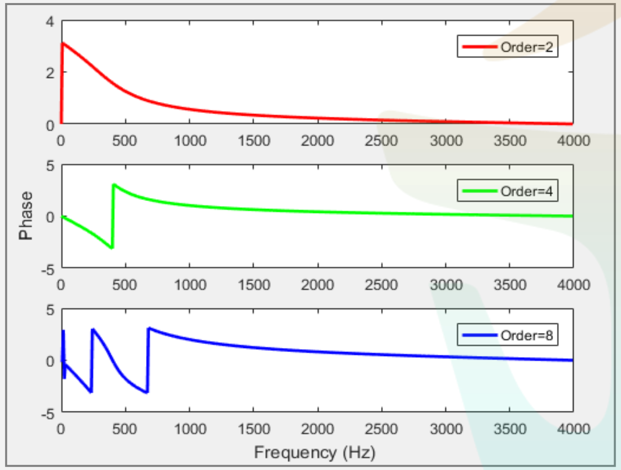


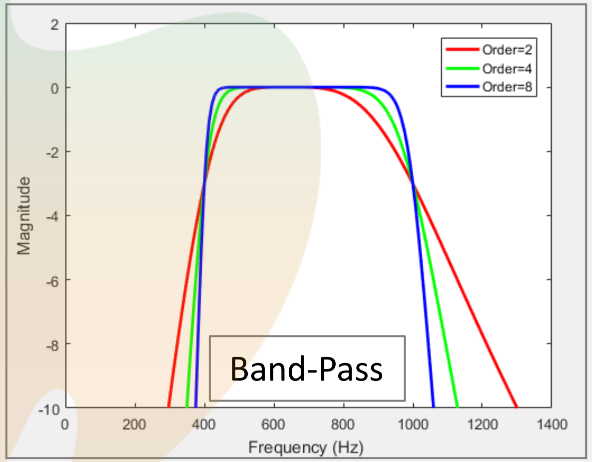
2. Low-Pass/High-Pass/Band-Pass

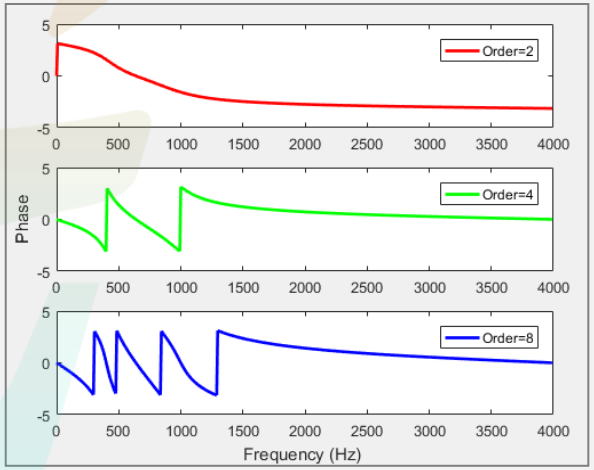








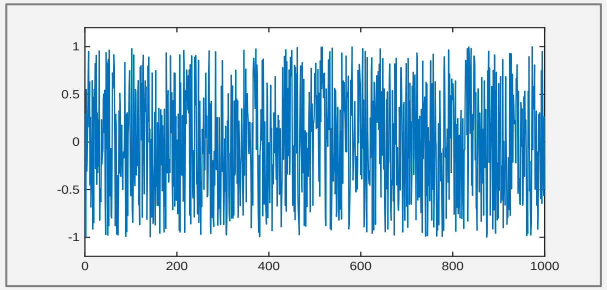


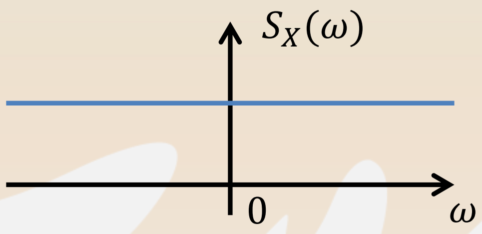


3.1 To Generate speech-shaped noise

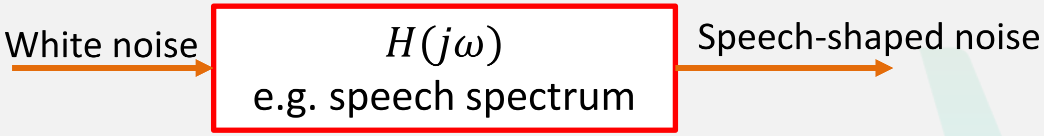
- **Generate speech-shaped noise**
  - White noise:

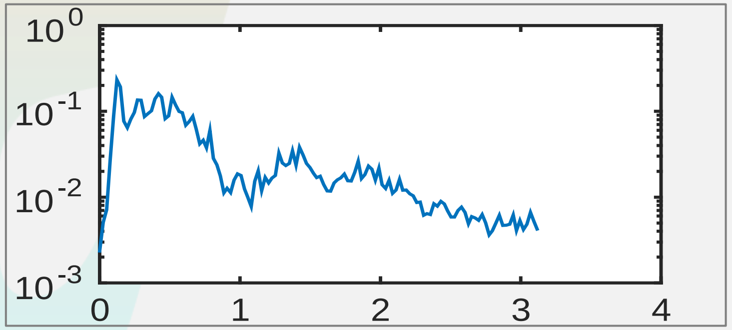
```
N=1000; noise = 1-2*rand(1,N);
```





  - Speech-shaped noise：





3.2 Steps to Generate speech-shaped noise

(a) Generate long-term spectrum of speech signal

- concatenate 10 speech waveforms into one signal
  - sig =[x,x,x,x,x,x,x,x,x,x]; % x is a row vector, what if x is a column vector?
  - Or, **repmat**(x,1,10)
- estimate the power spectral density of the speech signal
  - [Pxx,w] =**psd**(sig,512,fs); Or [Pxx,w] =**pwelch**(sig, [], [], 512, fs)

(b) Generate filter coefficients

- b = **fir2**(3000,w/(fs/2),sqrt(Pxx/max(Pxx))); % fir2: from frequency response to coefficients
- [h,wh] = freqz(b,1,128); %check frequency response here

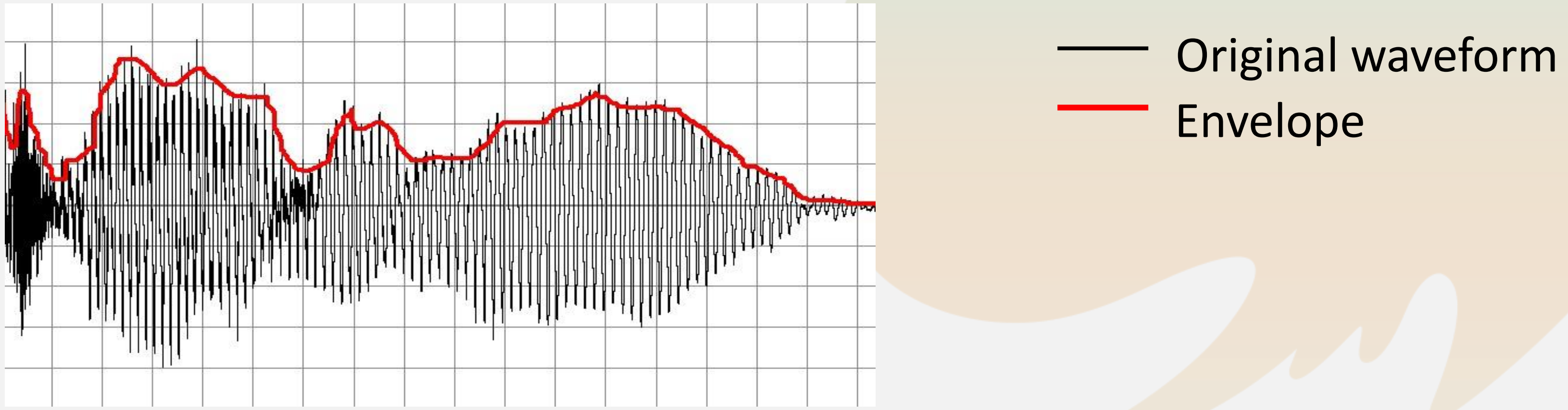
(c) Perform filtering for white noise signal

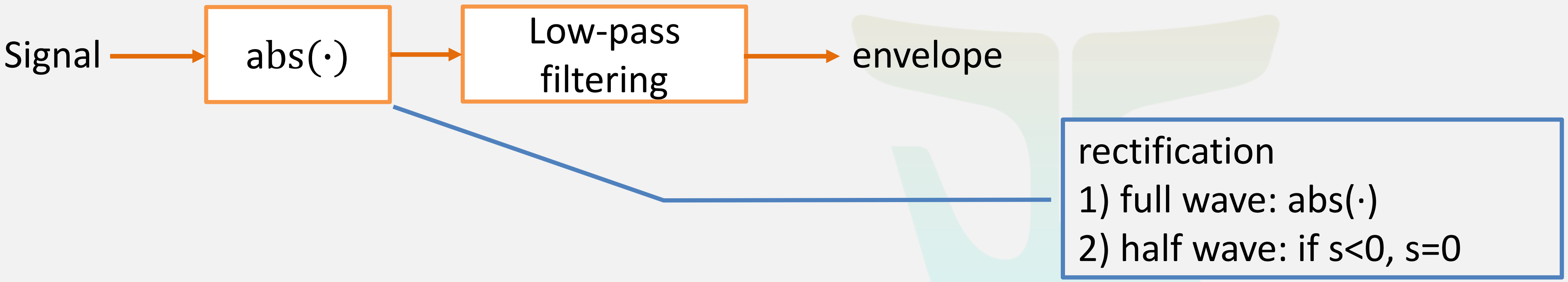
- y= filter(b,1,noise);

(d) Then you get the speech-shaped noise

3.3 To Extract speech envelope waveform

- **Extract speech envelope waveform**





4. Assignment

1. Generate a speech-shaped noise (SSN), and plot the spectra of the speech signal and SSN (e.g., use Matlab function ‘psd’ or ‘pwelch’, or other power spectral density estimation functions).
2. Read a speech signal x(t), adjust the SNR (x(t) to the above SSN) to -5 dB, let $y = x + SSN$, and normalize the energy of y in relative to x(t), i.e., modify the energy of y so that it equals to the energy of x.
3. Extract speech envelope

   1) with 2^{nd}-order low-pass filter and cutoff frequency f_{cut}= 100, 200, and 300 Hz. Plot these three envelope waveforms in one plot, and describe the difference among them.

   2) with 2^{nd} and 6^{th}-order low-pass filter and cutoff frequency 200 Hz. Plot these two envelope waveforms in one plot, and describe the difference between them.
4. One *.wav files for lab 5 and the coming lab project 1: ‘C_01_02.wav’

Tips: How to Adjust Signal Intensity

- **Signal-to-noise ratio (SNR)**
  - $SNR = 10log_{10}\left(E_{signal}/E_{noise}\right)$
  - *E* is energy, and *SNR* in dB scale
  - In Matlab: SNR=20*log10(norm(sig)/norm(noise)) % sig and noise must have the same length

n = norm(X) returns the 2-norm of input X and is equivalent to norm(X,2). If X is a vector, norm(X) is the **Euclidean distance**.

$\sqrt{E}$

- **Energy normalization**
  - y'=y*norm(x)/norm(y); % then y has the same energy as x
- **How to adjust SNR when adding noise to a signal?**
  - Adjust the energy of noise first
  - $E_{signal}/E_{noise} = 10^{\frac{SNR}{10}} \Rightarrow E_{noise} = f(SNR, E_{signal})$
  - After adjusting the noise energy, test if the SNR is the desired value.

Question ?